Query-Adaptive Hybrid Retrieval for Retrieval-Augmented Generation on Identifier-Heavy Corpora

Parth Tuteja
Dept. of Computer Science and Engineering
Chitkara University
Punjab, India
2310991172

Reetika
Dept. of Computer Science and Engineering
Chitkara University
Punjab, India
2310991267

Rishabh Puri
Dept. of Computer Science and Engineering
Chitkara University
Punjab, India
2310991269

***Abstract*—Retrieval-augmented generation (RAG) can only be as good as the passages it retrieves. We build a local hybrid RAG pipeline (structural chunking, BM25 and HNSW dense indexes, weighted reciprocal rank fusion, cross-encoder reranking and a 7B generator) and test whether the fusion weight can be predicted per query from cheap features. Our Query-Adaptive Fusion (QAF) uses a ridge regression over 12 query and score statistics, adds 0.3 ms per query and needs no LLM call. We evaluate on SciFact, FiQA, HotpotQA and a new set of 101 verified questions over 20 IETF RFCs, with BGE-small and EmbeddingGemma 2. Fixed-weight fusion beats the best single retriever on FiQA with both encoders and on the other collections with BGE-small, but not when the dense model is much stronger (EmbeddingGemma 2 on SciFact and HotpotQA) or when the weight was tuned on other collections (RFC, where BM25 alone is stronger). QAF does not reliably beat a dev-tuned fixed weight ($+0.020$ nDCG@10 on RFC, confidence interval including zero; no reliable gain elsewhere), although a per-query oracle shows $+0.167$ of headroom there before reranking. The features explain little of the oracle weight. Cross-encoder reranking gives the largest gains on HotpotQA and RFC but lowers scores on strong first stages and accounts for almost all of the latency. Better retrieval did not always mean better answers: with a local 7B generator, dense retrieval gave the best answers on HotpotQA, while the hybrid configurations did on RFC.**

***Index Terms*—retrieval-augmented generation, hybrid retrieval, reciprocal rank fusion, query-adaptive fusion, BM25, dense retrieval, cross-encoder reranking**

I. Introduction

Large language models answer fluently, but what they know is fixed at training time. They have not seen documents written after their cutoff or anything kept inside an organisation, and when a question falls outside what they learned they tend to produce a confident wrong answer [1]. Retrieval-augmented generation (RAG) [2], [3] fetches relevant passages at query time and asks the model to answer from them, which moves the burden onto the retriever: if the passage that holds the answer is not retrieved, the generator cannot recover it.

The two common retriever families fail in different places. Dense retrievers [4] match meaning and handle paraphrase well, but they often miss exact strings that were rare in their training data, such as version numbers or header names. A question about a status code can be answered from one line of one specification, and to an embedding model that line may look much like dozens of other passages on the same topic. BM25 [5] finds such strings easily, but it fails when the question and the source use different words. Hybrid retrieval runs both and merges the results, usually with Reciprocal Rank Fusion (RRF) [6], with one constant balance between the two sides. That constant is a compromise between queries that need exact matching and queries that need semantic matching.

Choosing the balance per query is not a new idea. DAT [7] asks an LLM to judge the top result of each retriever and sets the weight from those judgements. That works, but it puts LLM calls on the retrieval path of every query. We ask whether cheap signals from the query text and the two candidate lists are enough. Our method, Query-Adaptive Fusion (QAF), feeds 12 such features to a ridge regression that outputs the fusion weight. It is trained once on development queries, needs no LLM at query time, and its coefficients can be read off directly. In our experiments the answer is mostly no. QAF is cheap and readable, but it does not reliably beat a dev-tuned fixed weight, and we report this result together with an analysis of why.

We build a complete pipeline, from structural chunking through BM25 and dense indexing, weighted RRF and cross-encoder reranking to a local 7B generator. Everything runs on a single laptop. We test it on three public collections and on a new corpus of 20 IETF RFCs, with two encoders: BGE-small and Google's recent EmbeddingGemma 2. Our contributions are:

- **QAF**, an LLM-free, interpretable predictor of the per-query dense/sparse fusion weight, trained on a plateau-centre regression target derived from per-query weight sweeps, together with an analysis of why it does not reliably beat a tuned fixed weight even though per-query headroom exists.
- **An identifier-heavy RFC test collection** of 2,973 chunks and 101 questions, generated with a local LLM, filtered automatically and checked by all three authors, with

relevance labels derived from answer spans.

- **A controlled comparison** of sparse, dense, fixed-weight hybrid and QAF retrieval, with and without reranking, on four collections and two encoders, with paired bootstrap tests. It shows that fusion helps unless the dense model is much stronger or the weight was tuned elsewhere, that BM25 is the strongest single retriever on identifier-heavy RFC text, and that reranking can hurt a strong first stage. It includes an ablation and EmbeddingGemma 2 at four embedding sizes.
- **An end-to-end account** of latency (reranking, QAF and total) and answer quality (exact match, F1, citation accuracy) on a laptop with no paid API, showing that better retrieval does not always give better answers.

Sections II–IV cover related work, method and setup; Sections V–VII give results, discussion with limitations, and conclusions.

II. Related Work

Sparse and dense retrieval. BM25 [5] remains a strong baseline; on BEIR [8] it beats many neural retrievers outside their training domain. Dense retrievers address its vocabulary mismatch: DPR [4] trained a BERT bi-encoder that outperformed BM25 on open-domain QA, Sentence-BERT [9] made such encoders easy to train, and ColBERT [10] keeps one vector per token to recover some exact matching at the cost of a larger index. Among general-purpose embedders, BGE [11] is small and strong on English, while EmbeddingGemma [12] and its multimodal successor EmbeddingGemma 2 [13] target on-device use and are trained with Matryoshka representation learning [14], so their embeddings can be truncated.

Hybrid retrieval and fusion. Reciprocal Rank Fusion [6] merges ranked lists using ranks alone. Kuzi et al. [15] showed that lexical and semantic matching find different relevant documents, so running both raises recall, and Pyserini [16] made sparse, dense and hybrid runs reproducible. Bruch et al. [17] found that a tuned convex combination of normalised scores can beat RRF and that RRF is sensitive to its parameters. All of these use one setting for every query. DAT [7], the closest work to ours, asks an LLM to rate the top-1 dense and sparse results of each query and turns the ratings into the fusion weight. It reports gains over fixed weighting, at the price of LLM calls on the retrieval path. QAF targets the same decision with a linear model over cheap statistics and no model call at query time.

Reranking and RAG. A cross-encoder runs a full transformer pass per query–passage pair, so its cost grows with the number of candidates, whereas a bi-encoder embeds passages once, offline; Nogueira and Cho [18] therefore applied BERT only to a short first-stage list, the pattern we follow. DrQA [19] paired a TF-IDF retriever with a neural reader, RAG [2] trained a dense retriever and generator together, and Gao et al. [3] survey later variants. Ma et al. [20] rewrite queries before retrieval, and RAGAS [21] scores faithfulness with an LLM judge. PageIndex [22] drops both indexes and lets an LLM navigate a table-of-contents tree of each document, which suits structured texts such as RFCs but costs several LLM calls per query.

III. Method

Fig. 1 shows the pipeline. Each passage is indexed twice, in BM25 and in a dense vector index. A query goes to both retrievers, their top-100 lists are merged by weighted Reciprocal Rank Fusion (RRF) with a weight that is either constant or, in our method, predicted per query, and a cross-encoder reorders the head of the fused list before the top five passages reach the generator.

A. Chunking

BEIR and HotpotQA passages are already short and are indexed as they are. Chunking matters for the RFC corpus, where one document can run past 100 pages. Our chunker splits the cleaned text into blocks at blank lines. A one-line block is a heading if it is a numbered title of at most 12 words that does not end in a period, colon or comma ("4.2.1 Frame Header"), or a short line of at most 8 words that starts with a capital and contains only letters and spaces. A heading closes the current chunk and becomes the first line of the next, so section names are indexed with their bodies. Paragraphs are added until the chunk would pass 300 words (hard ceiling 380); longer paragraphs are split at sentences, then into word windows. Consecutive chunks of the same section share 50 words of overlap. Each chunk is a verbatim slice of its source and stores its character offsets, which we use for relevance labels. An optional break on low embedding similarity between paragraphs was off in every reported run, so boundaries follow document structure only.

B. Sparse retrieval

BM25 scores the title and text of each passage with the Lucene variant implemented in `bm25s` [23]:

$$s_{\mathrm{BM25}}(q,d) = \sum_{t \in q} \mathrm{idf}(t)\, \frac{f(t,d)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \quad (1)$$

where $f(t,d)$ is the frequency of t in d and $\mathrm{idf}(t) = \ln\!\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$ for N passages, n_t of which contain t [5]; $k_1 = 1.2$, $b = 0.75$.

The tokenizer is where we departed most from defaults. The default `bm25s` tokenizer keeps only runs of two or more word characters, so `HTTP/2` becomes `http` and the version number, often exactly what an RFC question asks about, is lost. Ours lowercases the text and first matches compounds, alphanumeric runs joined by `.`, `_`, `/`, `:` or `-`, and emits each compound both whole and as its parts (`http/2` gives `http/2`, `http`, `2`). Purely alphabetic tokens go through an English stopword list and the Snowball stemmer; tokens with a digit or separator are never stemmed or dropped. The same function serves documents, queries and the IDF look-ups used by QAF.

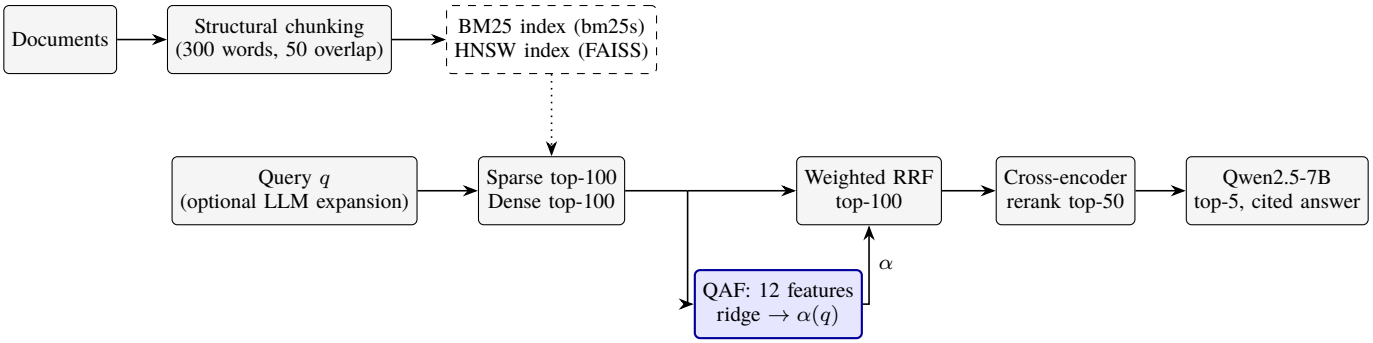


Fig. 1. Pipeline. Top: offline indexing. Bottom: query time. The shaded block is Query-Adaptive Fusion (QAF), which sets the dense weight α of the fusion step separately for every query.

C. Dense retrieval

We compare two bi-encoders: BGE-small-en-v1.5 [11], [24] (384 dimensions) and EmbeddingGemma 2 [13], the successor of EmbeddingGemma [12], whose 768-dimensional vectors are trained with a Matryoshka objective [14] so that prefixes of 512, 256 or 128 dimensions remain usable. We load only the text tower of EmbeddingGemma 2, in 32-bit floats. Each model gets its documented prompts: BGE prefixes queries with "Represent this sentence for searching relevant passages:" and encodes passages as plain text, while EmbeddingGemma 2 uses "task: search result | query:" for queries and "title: {title or none} | text: {passage}" for passages. Vectors are L2-normalised and stored in a FAISS [25] HNSW graph [26] ($M = 32$, *efConstruction* $= 200$, *efSearch* $= 128$) searched by inner product.

D. Weighted reciprocal rank fusion

BM25 scores and cosine similarities are on different scales, so we fuse ranks [6]. For a dense weight $\alpha \in [0, 1]$,

$$s_\alpha(d) = \frac{\alpha}{k + r_{\text{dense}}(d)} + \frac{1 - \alpha}{k + r_{\text{sparse}}(d)}, \qquad k = 60, \quad (2)$$

where $r(\cdot)$ is the 1-based rank in each top-100 list and a passage absent from a list gets nothing from it. The fused list keeps 100 passages, with ties broken by passage id. At $\alpha = 0.5$ this is plain RRF up to a constant factor; $\alpha = 1$ and $\alpha = 0$ give back the dense and sparse rankings.

E. Query-Adaptive Fusion (QAF)

One α for all queries is a compromise. "Which opcode marks a WebSocket close frame?" needs the sparse side; a loosely worded finance question needs the dense side. QAF predicts α per query from 12 features (Table I) that reuse work the pipeline has already done. Fig. 2 shows how the features are grouped, how α is produced, and how the model is trained.

Training target. For a labelled training query we sweep α over $\{0, 0.1, \ldots, 1\}$ and compute nDCG@10 of the fused list at each value. Since Eq. (2) uses ranks only, nDCG is piecewise constant in α and the optimum is usually a run of equally good grid points. Taking the first or last maximiser would push targets toward the ends of the range for no real reason, so the oracle $\alpha^o(q)$ is the centre of the longest contiguous run of grid values that reach the best nDCG@10; between equally long runs we keep the one nearer 0.5. A query with nDCG@10 of zero at every α carries no signal and gets 0.5.

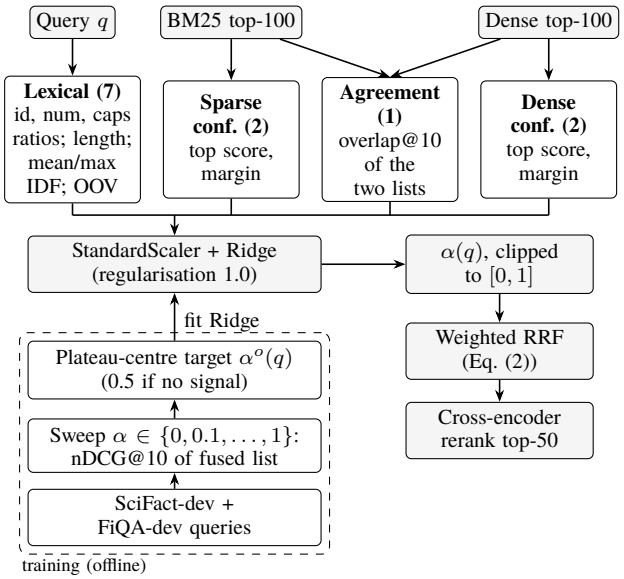


Fig. 2. QAF at query time (top and right) and its offline training (dashed box). The four feature groups together hold the 12 features of Table I.

Model and cost. Standardised features feed a ridge regression (regularisation 1.0) whose output, clipped to $[0, 1]$, is used as α. We fit one model per encoder on the pooled SciFact and FiQA dev queries; the RFC and HotpotQA collections are never seen in training. We chose a linear model so that its coefficients can be read and checked against intuition. Every feature comes from the query, the two candidate lists, the BM25 document frequencies or, for `oov_ratio`, the encoder's tokenizer vocabulary, so QAF adds feature extraction and a 12-term dot product, with no model call.

F. Reranking, post-processing and generation

The first 50 passages of the fused list are rescored by the BGE-reranker-base cross-encoder, which reads query and

TABLE I
QAF QUERY FEATURES. THE QUERY IS SPLIT INTO ALPHANUMERIC TOKENS (COMPOUNDS KEPT WHOLE); "RATIO" MEANS THE SHARE OF THOSE TOKENS.

Feature	Definition
`id_ratio`	identifier-like tokens: letters mixed with digits, an internal `_-./`, or all capitals
`num_ratio`	tokens made only of digits
`caps_ratio`	all-capital tokens of length ≥ 2 (acronyms)
`len_tokens`	number of tokens
`mean_idf`, `max_idf`	BM25 IDF of the query terms in the target corpus
`oov_ratio`	tokens absent from the dense encoder's tokenizer vocabulary (whole-word entries)
`sparse_top`, `sparse_margin`	top BM25 score and its gap to the second
`dense_top`, `dense_margin`	top cosine score and its gap to the second
`overlap_at10`	share of the dense top-10 also in the sparse top-10

TABLE II
COLLECTION STATISTICS (MEAN LENGTHS IN WHITESPACE-SEPARATED WORDS; RELEVANT PASSAGES PER EVALUATED TEST QUERY).

Dataset	Doc. words	Query words	Rel./query	Dev queries
SciFact	214.6	12.5	1.13	162
FiQA	139.3	10.9	2.63	500
HotpotQA	92.6	15.8	2.00	–
RFC	184.4	9.9	1.08	–

passage (title and text, up to 512 tokens) jointly as in monoBERT [18]; the top five passages of the new order form the context. The interactive pipeline can also drop near duplicates (Jaccard similarity of word 5-gram sets ≥ 0.9) and apply a metadata filter before reranking; both are disabled in the reported experiments. Optional query expansion, available in the demo but not evaluated in this paper, asks the local LLM for at most 20 keywords or synonyms and appends them to the query.

Answers come from Qwen2.5-7B-Instruct [27] served by Ollama at temperature 0 with a fixed seed. The prompt numbers the passages `[1]`–`[5]` and asks for a short answer citing them in that form, or the exact reply "not in context" when they do not support one. Cited numbers are mapped back to passage ids, so we can check whether a cited passage is relevant.

IV. EXPERIMENTAL SETUP

A. Datasets

We use four test collections: SciFact (5,183 passages, 300 test queries), a FiQA subset (20,000; 648), HotpotQA distractor paragraphs (4,955; 500) and our RFC set (2,973 chunks; 101). Table II gives dev-query counts, lengths and relevant passages per query; dev queries only fit QAF and choose α^*.

SciFact [28] and *FiQA* [29] come from BEIR [8]. SciFact has no dev split, so we hold out a seeded 20% of its training queries. To keep encoding time manageable we index 20,000 of FiQA's 57,638 posts: every post judged relevant for a dev or test query (2,944) plus a seeded random sample of

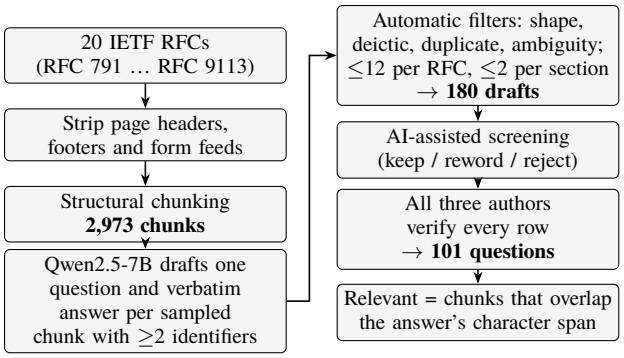


Fig. 3. Construction of the RFC test set. The filters run while questions are generated; generation stops once 180 drafts have passed them.

the rest. FiQA scores are therefore not comparable with full-corpus BEIR numbers. *HotpotQA* [30]: 500 seeded questions from the distractor validation set; the corpus is the union of their paragraphs, deduplicated by title, and the supporting paragraphs are relevant. It also supplies gold answers for the generation experiments.

RFC corpus. This collection targets questions about exact identifiers; Fig. 3 summarises how it was built. It holds 20 IETF RFCs, from IP and TCP (RFC 791, 793) through DNS, HTTP/1.1, URI syntax, IPv6, SMTP, WebSocket, OAuth 2.0, JWT, JSON and TLS 1.3 to QUIC and the 2022 HTTP documents (RFC 9110, 9112, 9113). After removing page headers and footers, the chunker of Section III-A yields 2,973 chunks. Each question's answer (at most eight words) is copied verbatim from its chunk; the filters reject, among others, answers that occur more than once in their chunk or more than three times in the parent RFC, and all three authors checked every surviving draft against the RFC text. A typical question asks for the length of the interface identifier in IPv6 unicast addresses ("64 bits"). A chunk is relevant if its character span overlaps the answer span.

B. Configurations

For every collection and encoder we run **BM25** and **Dense** alone; **Hybrid-fixed**, i.e. Eq. (2) with one weight α^*; **QAF**; both hybrids followed by reranking (**+RR**); and an **Oracle** that uses each query's plateau-centre oracle α (Section III-E) computed on the test labels. Because the centre of a plateau can fall between grid points, this is not a strict upper bound; we use it as a reference for how much headroom per-query weighting has, and report QAF's gap to it. α^* is the grid value with the best mean nDCG@10 over the pooled SciFact and FiQA dev queries (ties go to the value nearest 0.5) and is applied unchanged to all test sets ($\alpha^* = 0.9$ for BGE, 0.9 for EmbeddingGemma 2). We also sweep the fixed α over the grid on every test set. The ablation, on SciFact and RFC with EmbeddingGemma 2, removes one component at a time from the full system (structural chunks, QAF, reranking): QAF is replaced by α^*, reranking is dropped, and on RFC the structural chunks are replaced by fixed 256-word windows, with relevance re-derived from answer-span overlap. Query expansion

is implemented but was not evaluated. For the embedding-size study we truncate the stored EmbeddingGemma 2 vectors to 512, 256 and 128 dimensions, renormalise them, rebuild the HNSW index with the same settings, and retrain QAF and re-select α^* on the pooled dev queries at each size. Answers are generated with EmbeddingGemma 2 for BM25, Dense, Hybrid-fixed+RR and QAF+RR on 200 sampled HotpotQA questions and all 101 RFC questions.

C. Metrics

Retrieval is scored with Recall@k ($k \in \{5, 10, 20\}$), MRR@10 and

$$\text{nDCG@}k = \frac{1}{\text{IDCG@}k} \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \tag{3}$$

where rel_i is the graded relevance at rank i and IDCG@k is the same sum for the ideal ordering [31]. A query with no returned passages scores zero rather than being dropped. Answers are scored by SQuAD-style [32] exact match (EM) and token F1 after lowercasing and removing punctuation and articles; the `[n]` citation markers are stripped first. We also record whether a cited passage is relevant ("Cite"). As a side analysis, RAGAS [21] faithfulness is scored for Dense and QAF+RR on the first 100 HotpotQA questions (by id), with Qwen2.5-7B as judge. Every gain is reported as a mean per-query difference in nDCG@10 with a 95% paired-bootstrap confidence interval (10,000 resamples, seed 42) and tested with a one-sided paired bootstrap and paired t-test at level 0.05.

D. Implementation and hardware

Everything runs on one Apple M5 MacBook with 32 GB of memory, with encoders and reranker on PyTorch's MPS backend, the LLM in Ollama, and no paid API. The code uses Python 3.12, sentence-transformers, FAISS, bm25s and scikit-learn. Encoding dominates offline cost: EmbeddingGemma 2 encodes about 3.0–8.9 passages per second and BGE-small 65–169, so encoding the FiQA subset with the larger model takes about 38 minutes. All fusion variants are derived from cached top-100 lists, which a test confirms match the end-to-end pipeline. All sampling uses seed 42.

V. Results

Tables III and IV give retrieval scores and paired tests. "Gain" means the mean per-query difference in nDCG@10; headline numbers use EmbeddingGemma 2, the encoder with the better pooled-dev score for Hybrid-fixed. Our BM25 reaches 0.682 nDCG@10 on SciFact, close to the BM25 figure reported for BEIR [8], which we take as a check that indexing and evaluation are sound.

A. Hybrid fusion against single retrievers

Fixed-weight fusion is significantly better than the stronger single retriever on FiQA with both encoders, and on SciFact and HotpotQA with BGE-small (second block of Table IV). FiQA shows that the two sides need not be equally strong: BM25 is far behind dense there, yet fusion still helps. With

TABLE III

Retrieval effectiveness on the test queries. Best deployable value per column in bold; †: significantly better than Hybrid-fixed+RR (one-sided paired bootstrap, $p < 0.05$). ‡: per-query oracle α on test qrels, a headroom reference, not a strict upper bound. BM25 does not depend on the encoder.

Config	nDCG@10		Recall@10		MRR@10	
	BGE-small	Gemma	BGE-small	Gemma	BGE-small	Gemma
SciFact						
BM25	0.682	0.682	0.814	0.814	0.647	0.647
Dense	0.695	**0.851**†	0.830	**0.944**†	0.657	**0.828**†
Hybrid-fixed	0.707	0.830†	0.843	0.941†	0.667	0.800†
Hybrid-fixed+RR	0.712	0.734	0.844	0.869	0.678	0.698
QAF	0.721	0.801†	0.850	0.927†	0.685	0.769†
QAF+RR	**0.725**†	0.729	**0.861**	0.864	**0.690**†	0.695
Oracle‡	0.810	0.913	0.916	0.968	0.782	0.900
FiQA						
BM25	0.334	0.334	0.416	0.416	0.397	0.397
Dense	0.483†	0.406	0.547†	0.489	0.566†	0.480
Hybrid-fixed	**0.494**†	0.431	**0.561**†	0.519	**0.579**†	0.501
Hybrid-fixed+RR	0.431	0.426	0.513	0.508	0.505	0.497
QAF	0.469†	**0.435**	0.543†	**0.531**†	0.550†	0.498
QAF+RR	0.427	0.430	0.506	0.509	0.502	**0.504**
Oracle‡	0.563	0.532	0.616	0.598	0.665	0.631
HotpotQA						
BM25	0.785	0.785	0.891	0.891	0.876	0.876
Dense	0.870	0.923	0.934	0.977	0.936	0.962
Hybrid-fixed	0.874	0.921	0.941	**0.979**	0.937	0.962
Hybrid-fixed+RR	0.926	**0.936**	**0.962**	0.973	**0.970**	**0.973**
QAF	0.857	0.881	0.941	0.959	0.924	0.945
QAF+RR	**0.927**	0.934	0.960	0.973	**0.970**	0.970
Oracle‡	0.916	0.950	0.962	0.987	0.970	0.984
RFC						
BM25	0.786	0.786	**0.941**†	**0.941**	0.736	0.736
Dense	0.599	0.694	0.748	0.827	0.557	0.659
Hybrid-fixed	0.637	0.733	0.792	0.866	0.593	0.696
Hybrid-fixed+RR	0.792	0.823	0.876	0.921	0.769	0.792
QAF	0.686	0.763	0.827	0.906	0.645	0.719
QAF+RR	**0.813**	**0.843**	0.906	**0.941**	**0.788**	**0.813**
Oracle‡	0.871	0.900	0.970	0.970	0.842	0.881

EmbeddingGemma 2, whose dense run is much stronger (0.851 on SciFact), fusion significantly lowers SciFact and is level on HotpotQA (same block). On RFC, BM25 is the best single retriever, well ahead of either dense encoder; Hybrid-fixed without reranking differs from it by -0.053 and from dense by $+0.039$. Fig. 4 shows why: SciFact and HotpotQA improve toward the dense end, FiQA peaks in the interior, and RFC peaks near the sparse end, so $\alpha^* = 0.9$, chosen on the SciFact and FiQA dev queries, sits on the wrong side of the curve for RFC. At low dense weights the RFC curve rises above BM25 alone, so the problem there is the transferred weight, not fusion itself.

B. Does query-adaptive fusion help?

Not reliably. With EmbeddingGemma 2 and reranking, QAF+RR minus Hybrid-fixed+RR is -0.005 on SciFact (95% CI $[-0.011, 0.001]$), $+0.004$ on FiQA ($[-0.005, 0.013]$), -0.002 on HotpotQA ($[-0.006, 0.002]$) and $+0.020$ on RFC ($[-0.003, 0.053]$, $p = 0.0643$). Only on RFC is the difference sizeable in QAF's favour, and even there the interval includes zero. With BGE-small, QAF+RR wins narrowly on SciFact (first block of Table IV) and is ahead on RFC. The per-query spread is wide on every collection (standard deviation

TABLE IV

PAIRED SIGNIFICANCE OF THE MAIN COMPARISONS (TREATMENT MINUS BASELINE, NDCG@10). CI: 95% PAIRED-BOOTSTRAP INTERVAL (10,000 RESAMPLES, SEED 42); p: ONE-SIDED, TREATMENT BETTER THAN BASELINE, RESOLUTION 10^{-4} (10,000 RESAMPLES). †: BOOTSTRAP $p < 0.05$. COMPARISONS 1–3 ARE SHOWN FOR EACH ENCODER. HEADLINE NUMBERS IN THE TEXT USE GEMMA, CHOSEN BY POOLED DEV NDCG@10 OF HYBRID-FIXED (NOT ON TEST RESULTS).

Dataset	ΔnDCG@10	95% CI	p (boot.)	p (t-test)	n
QAF+RR vs. Hybrid-fixed+RR					
SciFact (BGE-small)	$+0.013^{\dagger}$	$[-0.000, 0.029]$	0.0280	0.0405	300
SciFact (Gemma)	-0.005	$[-0.011, 0.001]$	0.9586	0.9559	300
FiQA (BGE-small)	-0.004	$[-0.009, 0.001]$	0.9296	0.9281	648
FiQA (Gemma)	$+0.004$	$[-0.005, 0.013]$	0.2282	0.2263	648
HotpotQA (BGE-small)	$+0.001$	$[-0.004, 0.006]$	0.4220	0.4143	500
HotpotQA (Gemma)	-0.002	$[-0.006, 0.002]$	0.8045	0.8042	500
RFC (BGE-small)	$+0.021$	$[-0.002, 0.052]$	0.0561	0.0735	101
RFC (Gemma)	$+0.020$	$[-0.003, 0.053]$	0.0643	0.0824	101
Hybrid-fixed vs. best single retriever					
SciFact (BGE-small)	$+0.012^{\dagger}$	$[0.000, 0.023]$	0.0225	0.0239	300
SciFact (Gemma)	-0.021	$[-0.037, -0.007]$	0.9983	0.9966	300
FiQA (BGE-small)	$+0.011^{\dagger}$	$[0.003, 0.019]$	0.0035	0.0039	648
FiQA (Gemma)	$+0.026^{\dagger}$	$[0.017, 0.034]$	<0.0001	<0.0001	648
HotpotQA (BGE-small)	$+0.004^{\dagger}$	$[0.001, 0.007]$	0.0126	0.0122	500
HotpotQA (Gemma)	-0.002	$[-0.005, 0.001]$	0.9227	0.9182	500
RFC (BGE-small)	-0.149	$[-0.224, -0.074]$	1.0000	0.9999	101
RFC (Gemma)	-0.053	$[-0.127, 0.018]$	0.9212	0.9189	101
+RR vs. no RR (QAF)					
SciFact (BGE-small)	$+0.004$	$[-0.024, 0.033]$	0.3899	0.3885	300
SciFact (Gemma)	-0.072	$[-0.101, -0.043]$	1.0000	1.0000	300
FiQA (BGE-small)	-0.042	$[-0.061, -0.023]$	1.0000	1.0000	648
FiQA (Gemma)	-0.005	$[-0.025, 0.014]$	0.7002	0.6951	648
HotpotQA (BGE-small)	$+0.070^{\dagger}$	$[0.056, 0.083]$	<0.0001	<0.0001	500
HotpotQA (Gemma)	$+0.053^{\dagger}$	$[0.041, 0.065]$	<0.0001	<0.0001	500
RFC (BGE-small)	$+0.127^{\dagger}$	$[0.068, 0.189]$	<0.0001	<0.0001	101
RFC (Gemma)	$+0.081^{\dagger}$	$[0.017, 0.147]$	0.0072	0.0089	101
Gemma vs. BGE (QAF+RR)					
SciFact	$+0.004$	$[-0.004, 0.012]$	0.1757	0.1703	300
FiQA	$+0.003$	$[-0.005, 0.011]$	0.2190	0.2172	648
HotpotQA	$+0.007^{\dagger}$	$[0.003, 0.011]$	<0.0001	0.0002	500
RFC	$+0.030$	$[-0.012, 0.076]$	0.0820	0.0920	101

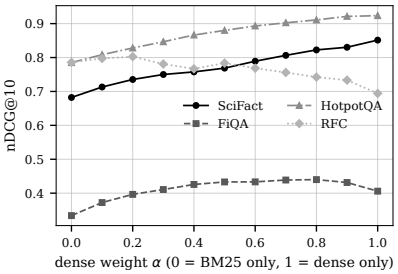


Fig. 4. nDCG@10 of fixed-weight fusion as a function of the dense weight α on each test collection (EmbeddingGemma 2, no reranking).

of nDCG@10 for QAF+RR: 0.362 on SciFact, 0.290 on RFC), so mean differences of this size are hard to separate from noise with our query counts.

There is room for a better weight. The plateau-centre oracle, which picks each query's weight from its own test labels, beats α^* by $+0.083$ on SciFact and $+0.167$ on RFC. It is a headroom reference, not an upper bound, and QAF recovers little of it.

C. What QAF learned

The ridge model explains little of the variance in the oracle targets: training R^2 is 0.06 on 662 pooled dev queries with EmbeddingGemma 2. Among the standardised coefficients, the largest belongs to `dense_top`: a confident dense retriever pushes the weight toward the dense side. Several features push it the other way, among them the BM25 margin, the overlap between the two lists and, less intuitively, the dense margin. The identifier share, which motivated the method, has a coefficient close to zero. As a result, predictions stay close to the intercept: on the test queries the predicted weight has mean 0.58 and standard deviation 0.07, and identifier-heavy queries get almost the same distribution as natural-language ones. In practice QAF behaves like a constant that has been shrunk from $\alpha^* = 0.9$ toward the middle of the range. That shift is probably what helps on RFC, whose sweep favours the sparse side, more than any recognition of identifier queries. The targets explain part of the difficulty: 39% of training queries have the same nDCG@10 at every grid weight and therefore carry the neutral target 0.5.

D. Reranking and latency

The cross-encoder is not a uniform win. Over QAF it adds large, significant gains on HotpotQA and RFC with both encoders (third block of Table IV), but it hurts strong first stages: -0.072 on SciFact with EmbeddingGemma 2, and a significant loss on FiQA with BGE-small. A general-purpose reranker over the top 50 can undo a good dense ranking when its notion of relevance differs from the collection's labels. EmbeddingGemma 2's dense SciFact score may also be helped by overlap between its training data and this public benchmark, which we cannot check.

Reranking dominates cost: with EmbeddingGemma 2 it takes 1186 ms of the 1206 ms mean per-query latency of QAF+RR, while the QAF decision takes 0.3 ms.

E. Statistical caveats

Table IV reports 28 one-sided tests without correction for multiple comparisons. Under a Bonferroni threshold of $0.05/28$, only the results with $p < 0.0001$ would survive: Hybrid-fixed over the best single retriever on FiQA with EmbeddingGemma 2, reranking on HotpotQA with both encoders and on RFC with BGE-small, and EmbeddingGemma 2 over BGE-small on HotpotQA. None of the QAF comparisons survive. FiQA scores come from a 20,000-passage subset that keeps every dev and test positive, so only relative comparisons on FiQA are meaningful.

F. Ablation

Table V removes one component at a time from the full EmbeddingGemma 2 system. Dropping the reranker raises SciFact, in line with the reranking results above, but costs RFC most of its gain. Replacing QAF by α^* changes little: SciFact moves slightly up and RFC slightly down. On RFC, fixed 256-word windows score clearly below structural chunks. This comparison changes two things at once, chunk boundaries and the size of the retrieval unit, and its relevance labels were re-derived from answer spans, so we read it as evidence for structure-aware chunking on RFC rather than as a clean measurement of it. Query expansion was not evaluated.

TABLE V
ABLATION OF THE FULL SYSTEM (EMBEDDINGGEMMA 2, THE DEV-SELECTED ENCODER; QUERY EXPANSION NOT EVALUATED). EACH ROW REMOVES ONE COMPONENT; BEST VALUE PER COLUMN IN BOLD.

Variant	SciFact		RFC	
	nDCG@10	Recall@10	nDCG@10	Recall@10
Full	0.729	0.864	**0.843**	**0.941**
−chunking	–	–	0.777	0.921
−QAF	0.734	0.869	0.823	0.921
−rerank	**0.801**	**0.927**	0.763	0.906

TABLE VI
EMBEDDINGGEMMA 2 WITH MATRYOSHKA TRUNCATION: NDCG@10 (BEST PER COLUMN IN BOLD). INDEX MB: DENSE FAISS HNSW FILE (VECTORS PLUS GRAPH), MEAN OVER THE DATASETS SHOWN. QUERY MS: MEAN DENSE-RETRIEVAL LATENCY PER QUERY, MEAN OVER THE DATASETS SHOWN.

Dims	SciFact Dense	SciFact QAF+RR	RFC Dense	RFC QAF+RR	Index MB	Query ms
768	**0.851**	0.729	**0.694**	0.843	13.0	17
512	0.851	0.727	0.669	0.843	9.0	17
256	0.836	**0.730**	0.640	**0.852**	5.0	17
128	0.792	0.727	0.539	0.846	3.0	17

G. Embedding size

Truncating EmbeddingGemma 2 hurts dense retrieval gradually. From 768 to 256 dimensions dense nDCG@10 drops by 0.035 on average over SciFact and RFC, with RFC losing more, and the loss grows at 128 (Table VI). After fusion and reranking the curve is almost flat, so on these two collections a 256-dimensional index costs nothing measurable in the full pipeline. The index file shrinks less than the dimension does, because the HNSW graph, which does not depend on the dimension, is part of it. Dense query latency stays in the tens of milliseconds at every size (Table VI) and is dominated by query encoding.

H. Answer quality

Answer quality does not follow retrieval quality on HotpotQA (Table VII). There, Dense gives the best answers and is significantly better in F1 than Hybrid-fixed+RR, although the reranked hybrids retrieve better. On RFC the two hybrid configurations give the best EM, F1 and citation accuracy, ahead of Dense and BM25. QAF+RR is not significantly better than Hybrid-fixed+RR on either collection. In the RAGAS side analysis on HotpotQA, Dense answers are judged slightly more faithful than QAF+RR answers (0.70 against 0.66). The judge is the same 7B model that wrote the answers, so this comparison may suffer from self-preference and we give it little weight. A sheet of sampled answers for manual faithfulness labelling is provided with the code; the team has not yet completed it.

VI. DISCUSSION

A. Why per-query weighting is hard to learn

We expected identifier-heavy queries to want a low dense weight and cheap features to detect them. The first half holds at the collection level, but the identifier share carries almost no signal. Three factors likely contribute: 39% of training targets are uninformative because nDCG@10 does not change with the weight; plateau centres on a coarse grid of a rank-based fusion are noisy targets; and the model never saw text like RFC. In-domain labels or post-retrieval evidence, as in DAT [7], may close more of the headroom; pre-retrieval statistics alone did not.

TABLE VII
ANSWER QUALITY. EM/F1 AGAINST THE GOLD ANSWER; CITE: A CITED PASSAGE IS RELEVANT. BEST PER COLUMN IN BOLD; †: SIGNIFICANTLY BETTER THAN HYBRID-FIXED+RR (ONE-SIDED PAIRED BOOTSTRAP, $p < 0.05$).

Config	HotpotQA			RFC		
	EM	F1	Cite	EM	F1	Cite
BM25	0.170	0.328	0.710	0.228	0.492	0.683
Dense	**0.215**	**0.410**†	**0.855**	0.228	0.469	0.624
Hybrid-fixed+RR	0.180	0.364	0.830	**0.248**	0.534	**0.713**
QAF+RR	0.195	0.377	0.815	**0.248**	**0.535**	**0.713**

B. Cost and deployment guidance

Four practical points follow. Sweep α on a small labelled sample of the target collection before fixing it, since a weight chosen on other domains landed on the wrong side of the curve for RFC. Keep BM25 for identifier-heavy text even with a strong dense encoder. Measure the reranker before adopting it: on our laptop it multiplies latency many times over and lowered nDCG@10 where the first stage was already strong. QAF costs well under a millisecond, but on our evidence it does not replace tuning. Finally, a 256-dimensional EmbeddingGemma 2 index is a safe saving inside a reranked hybrid pipeline, but not for dense retrieval on its own.

C. Limitations

All experiments run on subsets (FiQA 20,000 passages, HotpotQA 500 questions with their own paragraphs as the corpus, RFC 101 questions), so absolute scores are not comparable with full-benchmark numbers (Section V). The RFC questions were drafted by Qwen2.5-7B, the same model that generates answers, and every row was verified by all three authors, but the set favours questions that a 7B model finds easy to write, namely short identifier lookups without multi-hop items. RFC 2616, 7230 and 9110/9112 are successive HTTP/1.1 specifications with overlapping text, so a near-duplicate passage from a superseded RFC counts as a miss, even though questions ambiguous across RFCs were removed.

QAF was fitted on scientific and financial dev queries only, and we compare it with a fixed weight and the oracle but not with an LLM-based scheme such as DAT, so we cannot say how much accuracy the LLM-free design gives up. Query expansion was implemented but not evaluated. The RAGAS judge is the same 7B model that wrote the answers and covers only two configurations on HotpotQA, and the manual faithfulness check is still pending, so we make no claim about

faithfulness beyond that side analysis. The 28 significance tests are uncorrected, and RFC has only 101 questions. Latencies come from one laptop and will differ elsewhere in absolute terms. We also cannot rule out that EmbeddingGemma 2 saw SciFact or HotpotQA text during training, which would flatter its dense scores there.

VII. Conclusion

We built a hybrid RAG pipeline that runs entirely on a laptop and asked whether the dense/sparse balance should be set per query. Fixed-weight fusion helps unless the dense model is much stronger or the weight was tuned on other collections; on identifier-heavy RFC text, BM25 beat the dev-tuned hybrid. Our LLM-free QAF model is cheap and readable but does not reliably beat a fixed weight, behaving mostly like a shrunk constant, although the oracle headroom is real. Answers from a local 7B generator do not simply track retrieval quality: hybrids gave the best answers on RFC, dense retrieval on HotpotQA. Next we plan to compare against LLM-based weighting such as DAT and to add post-retrieval features to the weight predictor.

Acknowledgment

We thank our supervisor, Dr Rajat Takkar, Department of Computer Science and Engineering, Chitkara University, for his guidance throughout this project.

References

[1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474, arXiv:2005.11401.

[3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[4] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[5] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.

[7] H.-L. Hsu, J. Wang, and J. Tzeng, "DAT: Dynamic alpha tuning for hybrid retrieval in retrieval-augmented generation," arXiv:2503.23013, 2025.

[8] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021, arXiv:2104.08663.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[10] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2020, pp. 39–48.

[11] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2024, pp. 641–649, arXiv:2309.07597 (2023).

[12] H. Schechter Vera, S. Dua, B. Zhang, D. Salz, R. Mullins *et al.*, "EmbeddingGemma: Powerful and lightweight text representations," arXiv:2509.20354, 2025.

[13] H. Schechter Vera, S. Dua, J. Andonov, S. Smoot *et al.*, "EmbeddingGemma 2: Unified and efficient multimodal representations," arXiv:2610.37364, 2026.

[14] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi, "Matryoshka representation learning," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, arXiv:2205.13147.

[15] S. Kuzi, M. Zhang, C. Li, M. Bendersky, and M. Najork, "Leveraging semantic and lexical matching to improve the recall of document retrieval systems: A hybrid approach," arXiv:2010.01195, 2020.

[16] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," in *Proc. 44th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2021, pp. 2356–2362.

[17] S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, pp. 1–35, 2023.

[18] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[19] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading Wikipedia to answer open-domain questions," in *Proc. 55th Annu. Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1870–1879.

[20] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting in retrieval-augmented large language models," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing*, 2023, pp. 5303–5315.

[21] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158.

[22] VectifyAI, "PageIndex: Vectorless, reasoning-based RAG," https://github.com/VectifyAI/PageIndex, 2025, software repository, accessed Nov. 2026.

[23] X. H. Lù, "BM25S: Orders of magnitude faster lexical search via eager sparse scoring," arXiv:2407.03618, 2024.

[24] Beijing Academy of Artificial Intelligence (BAAI), "bge-small-en-v1.5: Model card," https://huggingface.co/BAAI/bge-small-en-v1.5, 2023, released with the FlagEmbedding toolkit, https://github.com/FlagOpen/FlagEmbedding; accessed Nov. 2026.

[25] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[26] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[27] Qwen Team, A. Yang, B. Yang, B. Zhang, B. Hui *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[28] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, "Fact or fiction: Verifying scientific claims," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 7534–7550.

[29] M. Maia, S. Handschuh, A. Freitas, B. Davis, R. McDermott, M. Zarrouk, and A. Balahur, "WWW'18 open challenge: Financial opinion mining and question answering," in *Companion Proc. The Web Conference 2018*, 2018, pp. 1941–1942.

[30] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. 2018 Conf. Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.

[31] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[32] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. 2016 Conf.*

Empirical Methods in Natural Language Processing, 2016, pp. 2383–2392.